ESCUELA DE ADMINISTRACIÓN Y NEGOCIOS

# ERS V2

Evaluación 2 (30%) — DSY1104

## Tienda online El Rincón de la Tía Ceci

Integrantes:

**Carolina Nataly Hermosilla Baeza**

**Pedro Felipe Toledo Jaramillo**

Asignatura: DSY1104 — Evaluación 2 (30%)

Sección: completar

Docente: completar

Fecha: 26-10-2026

Versión 2.0

Proyecto: Transformación del frontend front-pos a Next.js

Asignatura: DSY1104 — Evaluación 2 (30%)

Integrantes: Carolina Nataly Hermosilla Baeza · Pedro Felipe Toledo Jaramillo

Sección: completar

Docente: completar

Fecha: 26-10-2026

Versión: 2.0

---

# 1. Introducción

Este documento describe los requisitos de la versión 2 del frontend de la tienda online El Rincón de la Tía Ceci, un minimarket de barrio ubicado en Maipú. El proyecto toma como base el repositorio front-pos, que contenía una propuesta de punto de venta en HTML, CSS y JavaScript, y la transforma en una aplicación React con Next.js.

La aplicación conserva la identidad visual y los textos del minimarket, incorpora el flujo de compra de la tienda pública, un panel administrativo con operaciones CRUD, persistencia mediante una API simulada con json-server, diseño responsivo con Bootstrap 5 y pruebas unitarias con Jasmine y Karma.

El template HTML de referencia fue el Anexo 3 de la evaluación. De él se tomaron las ideas de estructura, navegación, tarjetas de producto, carrito, checkout y panel administrativo, adaptadas a la marca y a los datos del minimarket.

# 2. Objetivos

## 2.1 Objetivo general

Construir una tienda online funcional en Next.js que permita a los clientes explorar productos, gestionar un carrito y completar una compra simulada, y que entregue a los administradores y vendedores las herramientas para gestionar productos, categorías, clientes, usuarios, órdenes y reportes.

## 2.2 Objetivos específicos

- Migrar las vistas del proyecto HTML a componentes reutilizables de React.
- Implementar un archivo JavaScript de acceso a datos con operaciones CRUD.
- Conectar la interfaz con una API simulada que persista los cambios.
- Incorporar Bootstrap para lograr un diseño responsivo.
- Agregar vistas nuevas: detalle de producto, categorías, ofertas, checkout, compra exitosa, compra fallida, panel administrativo y reportes.
- Incorporar interactividad: búsqueda, filtros, ordenamiento, carrito, formularios validados y cambios de estado.
- Configurar y ejecutar pruebas unitarias con Jasmine y Karma.

# 3. Alcance

### 3.1 Incluido

• Tienda pública: inicio, catálogo, detalle de producto, categorías, ofertas, carrito, checkout, compra exitosa, compra fallida, nosotros, blog, contacto, Puntos Tía Ceci, inicio de sesión y creación de cuenta.

• Panel administrativo: dashboard, punto de venta interno, órdenes, productos, categorías, clientes, usuarios, reportes y perfil.

• CRUD de productos, categorías, clientes y usuarios.

• Cambio de estado de órdenes y devolución de stock al cancelar.

• Persistencia con json-server y persistencia local del carrito y la sesión.

• Pruebas unitarias y de componentes con Jasmine y Karma.

• Documento ERS V2 y documento de cobertura de testing.

### 3.2 Excluido

• Pasarela de pago real. El pago es una simulación académica.

• Autenticación con JWT y encriptación de contraseñas.

• Envío de correos reales desde el formulario de contacto.

• Carga o recorte de imágenes de productos.

• Costos de despacho. El envío y el retiro son sin costo.

• Módulos administrativos de pagos, facturas, proveedores y notificaciones del proyecto original.

• Aplicación móvil nativa.

## 4. Requisitos funcionales

| Código | Requisito | Prioridad |
|---|---|---|
| RF-01 | El visitante puede ver la página de inicio con hero, categorías, ofertas, historia y ubicación. | Alta |
| RF-02 | El visitante puede explorar el catálogo y buscar por nombre, SKU o descripción. | Alta |
| RF-03 | El visitante puede filtrar por categoría y ordenar por precio. | Alta |
| RF-04 | El visitante puede ver el detalle de un producto con precio, oferta y stock. | Alta |
| RF-05 | El visitante puede agregar productos al carrito y modificar cantidades. | Alta |
| RF-06 | El carrito se conserva al recargar la página. | Alta |
| RF-07 | El visitante puede completar el checkout como invitado o con sesión iniciada. | Alta |
| RF-08 | El checkout permite despacho a domicilio o retiro en tienda, sin costo. | Alta |
| RF-09 | El checkout valida RUN, correo, teléfono, dirección y medio de pago. | Alta |
| RF-10 | Al confirmar la compra se crea la orden, el pago, el movimiento de inventario y se descuenta stock. | Alta |
| RF-11 | El sistema muestra una página de compra exitosa y una de compra fallida. | Alta |
| RF-12 | El visitante puede registrarse como cliente y crear su cuenta de acceso. | Media |

| | | |
|---|---|---|
| RF-13 | El visitante puede iniciar sesión con usuario y contraseña. | Alta |
| RF-14 | El administrador puede ver y editar productos, incluido su stock. | Alta |
| RF-15 | El administrador puede crear, editar y eliminar categorías sin productos asociados. | Alta |
| RF-16 | El administrador puede activar y desactivar usuarios y clientes. | Media |
| RF-17 | El administrador puede revisar órdenes y cambiar su estado. | Alta |
| RF-18 | El vendedor puede usar el punto de venta interno y consultar órdenes y clientes. | Media |
| RF-19 | El dashboard muestra total recaudado, órdenes, productos y stock crítico. | Media |
| RF-20 | Los reportes muestran ventas por día, productos más vendidos y alertas de stock. | Media |
| RF-21 | El cliente puede actualizar su contraseña desde el perfil. | Baja |
| RF-22 | El formulario de contacto valida los datos y muestra un mensaje de éxito. | Baja |
| RF-23 | Puntos Tía Ceci registra o actualiza los datos del cliente. | Baja |
| RF-24 | El blog muestra tres artículos y su página de detalle. | Baja |

## 5. Requisitos no funcionales

| Código | Requisito |
|---|---|
| RNF-01 | La interfaz debe ser responsiva en móvil, tablet y escritorio con Bootstrap 5. |
| RNF-02 | Todo el código de la aplicación debe estar en JavaScript y usar componentes funcionales. |
| RNF-03 | Los componentes deben recibir datos por props y manejar estado local solo cuando sea necesario. |
| RNF-04 | El acceso a datos debe concentrarse en un módulo JavaScript reutilizable. |
| RNF-05 | Los precios y totales deben mostrarse en pesos chilenos, sin redondear. |
| RNF-06 | Las rutas deben estar en español y ser descriptivas. |
| RNF-07 | Las acciones destructivas deben pedir confirmación. |
| RNF-08 | El panel administrativo debe estar protegido por sesión y por rol. |
| RNF-09 | La aplicación debe funcionar sin un backend real, usando json-server. |
| RNF-10 | Las pruebas deben poder ejecutarse con un comando documentado. |

## 6. Actores y roles

| Actor | Descripción | Permisos |
|---|---|---|

| | | |
|---|---|---|
| Visitante | Persona que navega la tienda sin sesión. | Ver catálogo, carrito, comprar como invitado, registrarse e iniciar sesión. |
| Cliente | Usuario registrado con rol Cliente. | Comprar con datos precargados, ver su perfil y cambiar su contraseña. |
| Vendedor | Usuario interno con rol Vendedor. | Dashboard, punto de venta, consultar órdenes y clientes, perfil. |
| Administrador | Usuario interno con rol Administrador. | Todo el panel, incluidos productos, categorías, usuarios y reportes. |

### 6.1 Credenciales de demostración

| Usuario | Contraseña | Rol |
|---|---|---|
| admin | 12345 | Administrador |
| vendedor | 123 | Vendedor |

Las cuentas de cliente se crean desde la página de registro.

## 7. Arquitectura de la solución

La aplicación usa el App Router de Next.js 16 con React 19. Las páginas y los componentes interactivos son componentes de cliente (use client) porque necesitan estado, efectos, localStorage y eventos del navegador. El layout raíz se mantiene como componente de servidor para cargar los estilos globales.

La estructura principal es:

- app/(tienda): layout público con navbar y footer.
- app/admin: layout administrativo con sidebar y control de sesión.
- components: componentes reutilizables (botones, tarjetas, formularios, tablas de resumen, sidebar).
- lib: acceso a datos, carrito, sesión, validaciones, formato, estados y lógica de ventas.
- data: datos semilla, regiones/comunas y artículos del blog.
- db.json: base de datos simulada que consume json-server.
- tests: pruebas de Jasmine y Karma.

### 7.1 Tecnologías

| Tecnología | Uso |
|---|---|
| Next.js 16.4.3 | Framework de React con App Router. |
| React 19.3.0 | Componentes y estado. |
| JavaScript | Lenguaje de toda la aplicación. |
| Bootstrap 5.3.8 | Grid, componentes y diseño responsivo. |
| Bootstrap Icons 1.13.2 | Iconografía. |
| json-server 0.17.4 | API REST simulada en el puerto 4000. |
| Jasmine 5 + Karma 6 | Pruebas unitarias y de componentes. |
| React Testing Library | Renderizado y eventos en las pruebas. |
| Babel + Webpack | Compilación de las pruebas. |

# 8. Mapa de rutas

## 8.1 Tienda pública

| Ruta | Vista |
|---|---|
| / | Inicio |
| /productos | Catálogo con búsqueda, filtro y orden |
| /productos/[id] | Detalle del producto |
| /categorias | Listado de categorías |
| /categorias/[id] | Productos de una categoría |
| /ofertas | Productos con precio de oferta |
| /carrito | Carrito de compras |
| /comprar | Checkout |
| /comprar/ok | Compra exitosa |
| /comprar/error | Compra fallida |
| /nosotros | Historia y datos de la tienda |
| /blog | Listado de artículos |
| /blog/[id] | Detalle del artículo |
| /contacto | Datos de contacto y formulario |
| /puntos | Registro de Puntos Tía Ceci |
| /iniciar-sesion | Login |
| /crear-cuenta | Registro de cliente |

## 8.2 Panel administrativo

| Ruta | Vista | Roles |
|---|---|---|
| /admin | Dashboard | Administrador y Vendedor |
| /admin/ventas | Punto de venta interno | Administrador y Vendedor |
| /admin/ordenes | Listado de órdenes | Administrador y Vendedor |
| /admin/ordenes/[id] | Detalle y cambio de estado | Administrador y Vendedor (solo lectura) |
| /admin/productos | CRUD de productos | Administrador |
| /admin/categorias | CRUD de categorías | Administrador |
| /admin/clientes | Gestión de clientes | Administrador y Vendedor (solo lectura) |
| /admin/usuarios | Gestión de usuarios | Administrador |
| /admin/reportes | Reportes | Administrador |
| /admin/perfil | Perfil y cambio de contraseña | Administrador y Vendedor |

# 9. Modelo de datos

La API simulada usa las siguientes colecciones en db.json:

| Colección | Campos principales |
|---|---|
| categorias | id, nombre |

| | |
|---|---|
| productos | id, sku, nombre, descripcion, precioBase, precioOferta, granel, categoria, imagen |
| stock | id, idProducto, cantidad, stockMinimo |
| clientes | id, run, dvrun, pnombre, snombre, appaterno, apmaterno, correo, telefono, activo |
| usuarios | id, nombreUsuario, contrasena, rol, idCliente, activo |
| metodosPago | id, nombre, activo |
| ordenes | id, numeroOrden, idCliente, status, montoTotal, fechaVenta, idMetodoPago, entrega, direccion, region, comuna, stockDevuelto, detalles |
| pagos | id, idOrden, monto, estado, estadoFlow, fechaCreacion, fechaPago, idMetodoPago |
| movimientos | id, idProducto, tipoMovimiento, monto, fechaMovimiento |

### 9.1 Datos iniciales

Los datos de productos, categorías, clientes, órdenes y medios de pago provienen de la base de datos del proyecto de API VentaRápida. Se usan exactamente estos productos:

| Producto | Categoría | Precio base | Precio oferta | Stock |
|---|---|---|---|---|
| Arroz Tucapel | Abarrotes | $1.900 | $1.710 | 398 |
| Coca-Cola 2lt | Bebidas | $1.570 | $1.413 | 40 |
| Tallarines | Abarrotes | $1.490 | $1.341 | 39 |
| Cloro | Limpieza | $1.250 | $1.125 | 60 |
| Manzana | Frutas y Verduras | $1.490 | $1.341 | 50.000 |

El descuento de oferta es del 10% exacto, definido para esta evaluación. Los precios se muestran en pesos chilenos y no se redondean.

## 10. API simulada y CRUD

json-server expone las colecciones en http://localhost:4000. El módulo lib/api.js concentra las funciones de acceso:

- listarCategorias, obtenerCategoria, crearCategoria, actualizarCategoria, eliminarCategoria.
- listarProductos, obtenerProducto, crearProducto, actualizarProducto, eliminarProducto.
- listarClientes, obtenerCliente, crearCliente, actualizarCliente, buscarClientePorRun.
- listarUsuarios, obtenerUsuario, crearUsuario, actualizarUsuario, buscarUsuario.
- listarStock, crearStock, actualizarStock, eliminarStock, buscarStockPorProducto.
- listarOrdenes, obtenerOrden, crearOrden, actualizarOrden, siguienteNumeroOrden.
- listarPagos, crearPago, actualizarPago, buscarPagoPorOrden.
- listarMetodosPago, listarMovimientos, crearMovimiento.

### 10.1 Correspondencia con la API real

| Frontend (json-server) | API VentaRápida |
|---|---|
| GET/POST/PUT/DELETE /productos | /api/v1/productos |
| GET/POST/PUT/DELETE /categorias | /api/v1/categorias |
| GET/POST/PUT /clientes | /api/v1/clientes |
| GET/POST/PUT /ordenes | /api/v1/ordenes |
| GET/PUT /stock | /api/v1/stock |
| GET/POST/PATCH /pagos | /api/v1/pago |
| GET /metodosPago | /api/v1/metodos-pago |
| GET/POST /movimientos | /api/v1/movimiento-inventario |

Las rutas se simplificaron para que json-server funcione sin rewrites. Los pagos se normalizaron para apuntar al id de la orden y usar el mismo monto de la orden; la base original usaba el número de orden y tenía registros históricos inconsistentes.

## 11. Persistencia

• Servidor simulado: db.json guarda productos, categorías, clientes, usuarios, órdenes, stock, pagos y movimientos. Los cambios sobreviven mientras json-server esté ejecutándose.

• Carrito: se guarda en localStorage bajo la clave carrito. Sobrevive a la recarga y se sincroniza entre componentes mediante el evento carrito-actualizado.

• Sesión: se guarda en localStorage bajo la clave sesion. Contiene el id, nombre de usuario, rol e id de cliente.

• Datos semilla: data/seed.js es la fuente de verdad y npm run seed regenera db.json. No se debe editar db.json a mano si se quiere conservar el estado inicial.

## 12. Flujo de compra

1. El cliente agrega productos al carrito.
2. En /comprar completa sus datos, elige despacho o retiro y el medio de pago.
3. El sistema valida los datos y el stock.
4. Si el cliente no existe, se crea; si existe, se actualizan sus datos.
5. Se crea la orden en estado PENDIENTE_PAGO.
6. Se crea el pago en estado PENDIENTE.
7. Se descuenta el stock y se registra un movimiento de salida.
8. Se vacía el carrito y se redirige a /comprar/ok.
9. La página de compra exitosa marca la orden y el pago como PAGADO.
10. Si el stock es insuficiente, se redirige a /comprar/error?motivo=stock.

El punto de venta interno del administrador usa el mismo servicio, pero crea la orden directamente en estado PAGADO.

## 13. Panel administrativo

• Dashboard: total recaudado, órdenes, productos, stock crítico, últimas órdenes y alertas.

• Nueva venta: catálogo, búsqueda, filtro, cliente por RUN, carrito, medio de pago y cobro.

• Productos: listado, búsqueda, creación, edición, eliminación y control de stock.

• Categorías: listado, creación, edición y eliminación; no permite eliminar categorías con productos.

• Órdenes: listado, filtro por estado, detalle y cambio de estado. Al cancelar se devuelve el stock una sola vez.

• Clientes: búsqueda, creación, edición y activación/desactivación.

• Usuarios: creación, edición, activación/desactivación y asociación a un cliente.

• Reportes: ventas por día, productos más vendidos, alertas de stock y tarjetas de resumen.

• Perfil: datos de la cuenta y cambio de contraseña.

## 14. Pruebas

Se configuró Karma con Jasmine, Webpack y Babel. Las pruebas cubren:

• Funciones de acceso a datos (lib/api.js).
• Operaciones del carrito (lib/carrito.js).
• Validaciones de RUN, correo, teléfono, textos y contraseñas.
• Componente Boton.
• Componente TarjetaProducto.
• Formulario FormularioProducto.

Resultado de la ejecución: 33 pruebas exitosas, 0 fallidas. El detalle está en el documento Cobertura-Testing.md.

## 15. Instalación y ejecución

```
npm install
npm run seed
```

En una terminal:

```
npm run mock
```

En otra terminal:

```
npm run dev
```

La tienda queda en http://localhost:3000 y la API simulada en http://localhost:4000.

### 15.1 Otros comandos

```
npm run build      # compilación de producción
npm run start      # servidor de producción
npm run lint       # revisión de ESLint
npm test           # Jasmine + Karma con ChromeHeadless
npm run test:jsdom # Jasmine + Karma con jsdom (alternativa sin Chrome)
```

## 16. Limitaciones y decisiones

• El login es una simulación: las contraseñas se guardan en texto plano y no hay JWT. No debe usarse en producción.

• El pago es simulado; no se integra una pasarela real.

• El catálogo usa los 5 productos de la API y una imagen de respaldo para los productos que no tienen fotografía.

• La orden 1989 del SQL original tiene un total histórico que no coincide con la suma de sus detalles; se conserva el dato original y se documenta.

• Tres clientes del SQL original tenían un dígito verificador de RUN que no correspondía al número; se corrigió el dígito para que la validación del formulario fuera coherente.

• Los pagos se normalizaron para que un pago corresponda a una orden.

• No se cobra despacho: el envío y el retiro son sin costo.

• No se implementan los módulos de pagos, facturas, proveedores y notificaciones del proyecto original.

• La sección y el docente quedan como “completar” porque no fueron informados.

• Los textos del blog fueron adaptados del blog de front-pos.

## 17. Conclusiones

La transformación permitió llevar una propuesta HTML de punto de venta a una aplicación Next.js organizada por componentes, con una tienda pública completa, un panel administrativo con CRUD y persistencia, y una base de pruebas automatizadas. La separación entre datos, lógica y presentación facilita explicar el proyecto y extenderlo en futuras evaluaciones.

---

Anexo A. Origen de los datos

• Repositorio base del frontend: https://github.com/KriegDev/front-pos

• Repositorio de la API: https://github.com/KriegDev/ventarapida-hermosilla-toledo-001d

• Template HTML de referencia: Anexo 3 de la Evaluación Parcial 2.

• Proyecto de clases de referencia: mi-tienda.